Universidad Rafael Landívar

Facultad de Ingeniería

Inteligencia artificial

Sección: 1

Catedrático: Max Cerna

PROYECTO

Clasificación de Solicitudes a Mesa de Ayuda

Alumnos: Giancarlo Ortega Bonilla

Carnet: 1230222

Mario Lopez Joaquin

Carnet: 2477123

Walter Herrera Salvatierra

Carnet: 1109722

Introducción

El presente proyecto desarrolla un sistema de clasificación automática de tickets de soporte al cliente utilizando el algoritmo de Naive Bayes. Este tipo de sistemas tiene una aplicación directa en empresas que reciben diariamente grandes volúmenes de solicitudes de clientes, abarcando temas como problemas con pedidos, consultas de facturación, solicitudes de reembolso, entre otros, donde gestionar todo esto de forma manual representa un desafío importante en términos de tiempo y recursos, por lo que son muy útiles el contar con esto, ya que sino seria mucha demora para el usuario e incluso se puede perder su confianza, afectando así en la empresa, tanto en clientes, como en dinero.

El énfasis de este proyecto está en comprender e implementar manualmente cada etapa del algoritmo, desde el preprocesamiento del texto mediante tokenización y stemming, la construcción del vocabulario a partir del corpus de entrenamiento, el cálculo de probabilidades condicionales usando el Teorema de Bayes, la aplicación de Laplace Smoothing para manejar palabras no vistas y la evaluación del modelo mediante K-Folds Cross Validation, por lo que en este proyecto no se utilizan librerías que resuelven el problema de clasificación de forma automática.

Para poder lograr llevarlo a cabo, se utilizó el dataset Bitext Customer Support LLM Chatbot Training Dataset, que contiene 26,872 muestras de tickets de soporte técnico reales distribuidas en 11 categorías: account, cancel, contact, delivery, feedback, invoice, order, payment, refund, shipping y subscription. Este conjunto de datos fue seleccionado por la coherencia semántica entre el contenido de los textos y sus etiquetas asignadas, característica fundamental para el éxito del aprendizaje supervisado. El sistema resultante es capaz de recibir una solicitud en texto libre, procesarla, y predecir automáticamente la categoría correspondiente junto con el nivel de confianza de la predicción.

El proyecto integra un motor de inferencia desarrollado en Python como backend, y una interfaz web funcional desarrollada con Flask, HTML y CSS como frontend, permitiendo al usuario ingresar una solicitud y visualizar la categoría predicha en tiempo real junto con las probabilidades por clase.

Definición del Problema y Objetivos

Al ponerse del lado de una empresa que ofrece servicios o productos en línea diariamente, son exageradas las cantidades de solicitudes que les llegan, ya sea de soporte y ayuda, e incluso pueden hasta ser de quejas. Por lo que llega a ser algo muy extenso y complejo. Si uno lo analiza bien el hacer todo esto de manera manual para poder decidir a dónde enviar la solicitud y que lo puedan resolver, seria muchisimo tiempo para el cliente, por lo que haría perder dinero a la empresa e incluso la confianza que se le puede tener, por lo que se debe de ver la manera de ayudarse de la tecnología y tenerlo asi mas automatizado.

Por lo que ese sería el desafío, el poder crear un tipo de modelo de clasificación de texto el cual puede llegar a funcionar como un tipo de cerebro digital, el cual apoye con la asignación de los tickets, por medio de aprender patrones de un conjunto de datos previamente etiquetado, siendo así capaz de poder generalizar sobre las nuevas solicitudes que vayan llegando al sistema.

Objetivo General
Crear un sistema inteligente de clasificación de las solicitudes de los tickets, el cual permita asignar correctamente una categoría a cada solicitud en función de su contenido textual, haciendo todo esto posible con su correcto entrenamiento y buen uso del set de datos que fue brindado para que esté listo.

Objetivos Específicos:

- Preprocesar los datos textuales, teniendolos listos y limpios para su uso
- Construir un modelo de representación de texto por medio de Bag of Words
- Implementar el algoritmo de Naive Bayes, usando cálculos de probabilidad a priori por clase, uso de Laplace para palabras no vistas, y uso de logaritmos para evitar underflow numérico
- Evaluar modelos usando K-Folds cross, con un K mínimo de 5
- Analizar y poder entender las métricas que son utilizadas en el proyecto, siendo estas: Precision, Recall, F1-Score
- Guardar y cargar el modelo entrenado desde un archivo persistente
- Desarrollar interfaz web para poder simular un portal de soporte al cliente, donde el usuario interactúa con la página y puede ingresar sus solicitudes, y así poder recibir la categoría predicha
- Integrar correctamente el backend con el frontend

Descripción del dataset utilizado

El dataset utilizado en este proyecto es el Bitext Customer Support LLM Chatbot Training Dataset, disponible públicamente en la plataforma Hugging Face. Se trata de un dataset híbrido sintético diseñado para el entrenamiento de modelos en el dominio de atención al cliente, generado mediante tecnología avanzada de procesamiento y generación de lenguaje natural junto con herramientas automatizadas de etiquetado de datos.

El dataset contiene aproximadamente 26,872 instancias, cada una representando una solicitud de cliente en formato de texto natural, acompañada de una etiqueta que indica la categoría correspondiente, cada uno de los registros está compuesto principalmente de los campos instruction, los cuales contienen el texto de solicitud, y category representa la clase asignada.

Una de las ventajas que se tiene de usar estos datos es que sus etiquetas son creadas de manera controlada y coherente con el contenido del texto. Esto permite que el modelo de clasificación aprenda patrones reales entre las palabras y las categorías, mejorando su capacidad de generalización frente a nuevos datos.

Asimismo, las solicitudes que el dataset presenta tienen una gran variedad lingüística, los cuales incluyen lenguaje informal y formal, también errores intencionales y distintos registros de escritura, esto resulta muy útil, ya que simula de forma realista como los clientes pueden llegar a redactar las solicitudes, en contextos reales, lo cual ayuda a entrenar un modelo más robusto y que sea más adaptable a las necesidades y las distintas formas de expresión.

En conjunto, estas características hacen que el dataset sea adecuado para entrenar y evaluar de manera efectiva el modelo de clasificación implementado en este proyecto.

Descripción del preprocesamiento aplicado

Antes de dirigirse a entrenar el modelo, cada solicitud del dataset es sometida a un proceso de limpieza de datos y normalización del texto, llegando así a reducir el ruido en los datos y al mismo tiempo transformar el texto en una representación que el clasificador pueda procesar eficientemente, dejando los datos listos para poder proceder al entrenamiento. Este proceso es implementado en la clase ProcesadorTexto y realiza lo siguiente:

- Se descargan los recursos de NLTK, el cual es una librería de python la cual es un conjunto de herramientas para poder trabajar con lenguaje natural, en este caso texto humano, con esto se puede tokenizar el texto, ayuda mucho el separar las palabras y frases, también eliminar palabras irrelevantes que no aportan mayor cosa

- Luego lo que se realiza es que se convierte todo el texto a minúsculas, esto para evitar que mismas palabras sean tratadas como términos distintos por tener alguna que otra letra en diferente tamaño(minuscula o mayuscula)

- Se remueven del dataset los placeholders tipo “order number” o “customer support hours”, ya que son tokens que no aportan mayor información semántica

- Se elimina el ruido, tales como las url, direcciones de correo electrónico, números y puntuación ya que no ayudan a identificar la categoría de una solicitud

- Todo el texto ya se tiene limpio, por lo que ahora se divide en tokens individuales, usando la función de nltk, word_tokenize

- Se eliminan las stopwords en inglés, esto se hace ya que aparecen muchas veces, pero no aportan ningún tipo de valor para la clasificación

- Se aplica el stemming, por lo que significa que cada token es reducido a su raíz, eliminando los sufijos o terminaciones, esto disminuye el tamaño del vocabulario y mejora la generalización del modelo

- Los tokens muy cortos son eliminados, más que nada los de 2 caracteres o menos, ya que generalmente corresponden a abreviaciones o residuos del proceso de limpieza, por lo que no tienen valor semántico

Gracias a este proceso, cada solicitud que llegue a entrar al sistema quedará representada como una lista de tokens normalizados, listos para su construcción del vocabulario y entrenamiento del modelo

Explicación matemática del algoritmo Naïve Bayes, Laplace Smoothing y suma de logaritmos

1. Algoritmo Naïve Bayes

El algoritmo Naïve Bayes es un clasificador probabilístico fundamentado en el Teorema de Bayes, que establece la relación entre la probabilidad condicional de un evento dado otro y sus probabilidades individuales. En el contexto de clasificación de texto, el objetivo es determinar la probabilidad de que un texto pertenezca a una clase específica dada la evidencia proporcionada por las palabras que lo componen.

Fórmula principal:

$P(C \mid X) = (P(X \mid C) * P(C)) / P(X)$

Donde:

- $P(C \mid X)$: Probabilidad de la clase dado el texto
- $P(X \mid C)$: Probabilidad del texto dado la clase
- $P(C)$: Probabilidad previa de la clase
- $P(X)$: Probabilidad del texto

En la práctica de clasificación, $P(X)$ actúa como una constante de normalización que es idéntica para todas las clases, por lo que puede omitirse al comparar las probabilidades posteriores de diferentes clases. Por tanto, el clasificador selecciona la clase que maximiza el numerador del teorema:

$C^* = \text{argmax} [P(X \mid C) * P(C)]$

donde C^* representa la clase predicha para C.

2. Supuesto de Independencia (Naïve)

El modelo asume que todas las palabras son independientes entre sí, lo que simplifica el cálculo.

$P(X \mid C) = P(w1 \mid C) * P(w2 \mid C) * ... * P(wn \mid C)$

Aunque no es completamente realista, funciona bien en clasificación de texto (Manning et al., 2008).

3. Modelo Multinomial

El modelo Multinomial de Naive Bayes es particularmente apropiado para clasificación de textos donde las características son frecuencias de términos. En este modelo, la probabilidad a priori de una clase se estima mediante la proporción de documentos de entrenamiento que pertenece a esa clase:

P(C) = número de documentos en la clase / total de documentos
P(w | C) = frecuencia de la palabra en la clase / total de palabras en la clase
Luego se calcula el score y se selecciona la clase con mayor probabilidad.

4. Laplace Smoothing
Un problema crítico del modelo básico de Naïve Bayes es que si una palabra del documento de prueba nunca apareció en los documentos de entrenamiento de una clase específica, entonces P(w|C) = 0, lo que causa que todo el producto de probabilidades condicionales sea cero independientemente de las demás palabras. Este fenómeno se conoce como el problema de probabilidad cero.
El Laplace Smoothing, también conocido como como suavizado aditivo, resuelve este problema agregando una constante α (típicamente α = 1) al conteo de cada palabra en el numerador y α * |V| al denominador, donde |V| es el tamaño del vocabulario.

Fórmula:
P(w | C) = (count(w, C) + α) / (total de palabras en la clase + α*tamaño del vocabulario)
Esta técnica garantiza que todas las palabras, incluso aquellas nunca vistas en una clase, tengan una probabilidad no nula.

5. Uso de Logaritmos
Para evitar problemas numéricos al multiplicar probabilidades pequeñas, se utilizan logaritmos.
Fórmula:
log P(C | X) = log P(C) + suma de log P(w_i | C)
Esto convierte multiplicaciones en sumas y mejora la estabilidad del cálculo.

6. Interpretación en el Proyecto
El modelo aprende qué palabras son más comunes en cada categoría y usa probabilidades para clasificar nuevas solicitudes.
Laplace Smoothing evita errores con palabras desconocidas y los logaritmos aseguran cálculos estables.

Explicación e interpretación de las métricas de evaluación

- Definición y fórmula de Precisión, Recall, F1-Score y Accuracy.

1. Accuracy (Exactitud)
El Accuracy mide el porcentaje total de predicciones correctas realizadas por el modelo.
Fórmula:
Accuracy = (TP + TN) / (TP + TN + FP + FN)
Indica el rendimiento global del modelo, aunque puede ser engañoso si hay clases desbalanceadas.

2. Precisión (Precision)
La precisión mide qué tan confiables son las predicciones positivas del modelo.
Fórmula:
Precision = TP / (TP + FP)
Indica cuántas predicciones positivas fueron realmente correctas.

3. Recall (Sensibilidad)
El recall mide qué tan bien el modelo detecta todos los casos reales positivos.
Fórmula:
Recall = TP / (TP + FN)
Indica cuántos casos reales fueron correctamente identificados.

4. F1-Score
El F1-Score combina la precisión y el recall en una sola métrica.
Fórmula:
F1 = 2 * (Precision * Recall) / (Precision + Recall)
Permite evaluar el equilibrio entre precisión y recall.

5. Interpretación en el Proyecto
Estas métricas permiten evaluar el rendimiento del modelo de clasificación de tickets.
Es importante analizarlas junto con la matriz de confusión para entender errores entre categorías.

- Interpretación de la Matriz de Confusión generada.

La matriz de confusión es una herramienta que permite visualizar el desempeño del modelo de clasificación comparando las predicciones realizadas con los valores reales.

La matriz de confusión permite ver el rendimiento del modelo comparando predicciones con valores reales. En este caso incluye 11 categorías (ACCOUNT, CANCEL, CONTACT, DELIVERY, FEEDBACK, INVOICE, ORDER, PAYMENT, REFUND, SHIPPING y SUBSCRIPTION). Las filas son los valores reales y las columnas las predicciones; la diagonal muestra aciertos y el resto, errores.

¿Cómo se interpreta?

- La diagonal principal contiene los aciertos del modelo.
- Los valores fuera de la diagonal representan errores de clasificación

Por ejemplo:

- Si muchos tickets de “Queja” aparecen clasificados como “Cancelación”, significa que el modelo está confundiendo esas dos categorías.
- Si una clase tiene valores altos en la diagonal, indica que el modelo la reconoce correctamente con frecuencia.

Importancia

La matriz de confusión permite:

- Identificar patrones de error
- Detectar clases que se confunden entre sí.
- Evaluar el comportamiento del modelo más allá del accuracy.

Esto es clave porque un modelo puede tener buena exactitud general, pero aún así fallar en clases específicas.

- Análisis crítico: ¿qué clases se predicen mejor y peor? ¿Por qué?

Generalmente, en este tipo de problemas, las clases que se predicen mejor son aquellas que:

- TIenen palabras clave muy específicas
- PResentan poco solapamiento semántico con otras categorías
- Tienen suficiente cantidad de datos en el dataset

En este proyecto, es probable que:

- Las clases con mejor desempeño fueron CONTACT, CANCEL y SUBSCRIPTION, debido a que tienen un vocabulario más específico y menos confusión con otras categorías. La de menor rendimiento fue DELIVERY (F1-Score de 95.45%), principalmente por su similitud con SHIPPING, ya que ambas comparten términos relacionados con envíos, lo que genera confusión en el modelo.

Clases que se predicen peor

- La clase con mayor dificultad de clasificación fue DELIVERY, principalmente por compartir vocabulario similar con SHIPPING, ya que ambas manejan términos relacionados con envíos y direcciones, lo que genera confusión entre ellas.

Ejemplo de confusión

Un ticket como:
“Quiero cancelar porque el servicio no funciona bien”

Esto genera errores porque el modelo no entiende el contexto completo, solo las palabras individuales.

Razones de los errores
Los principales factores que explican estos errores son:

a) Supuesto de Independencia (Naïve)
   El modelo no entiende el contexto completo de la frase, solo analiza palabras individuales, lo que puede llevar a interpretaciones incorrectas.

b) Ambigüedad del lenguaje
   Muchas solicitudes pueden pertenecer a más de una categoría dependiendo del contexto.

c) Desbalance de datos
   Si hay más ejemplos de una clase que otra, el modelo tiende a favorecer la clase más frecuente.

d) Palabras compartidas entre clases
   Palabras como “problema”, “servicio” o “solicitud” pueden aparecer en todas las categorías, lo que reduce la precisión.

Evaluación del desempeño general
Un buen modelo debe:

- Tener valores altos en la diagonal de la matriz
- Minimizar errores entre clases similares
- Mantener equilibrio entre precisión y recall

Sin embargo, en este tipo de problema, es más importante analizar:

- Qué errores son más frecuentes
- Si estos errores afectan la experiencia del usuario

Mejora del modelo

A partir del análisis de la matriz de confusión, se pueden proponer mejoras:

- Mejorar el preprocesamiento (lematización, limpieza más profunda)
- Aumentar el tamaño del dataset
- Balancear las clases
- Incorporar n-gramas (frases en lugar de palabras individuales)
- Ajustar el modelo para manejar mejor contexto

Conclusión del Análisis

La matriz de confusión no solo muestra si el modelo funciona, sino cómo y en qué se equivoca.

En este proyecto, el modelo Naïve Bayes puede ser efectivo para clasificar tickets con vocabulario claro, pero presenta limitaciones en categorías ambiguas o similares.

Por lo tanto, el análisis crítico permite entender que el rendimiento del modelo no depende únicamente de la exactitud, sino de su capacidad para diferenciar correctamente entre clases que comparten características similares.

Resultados obtenidos con K-Folds y análisis de varianza entre folds.

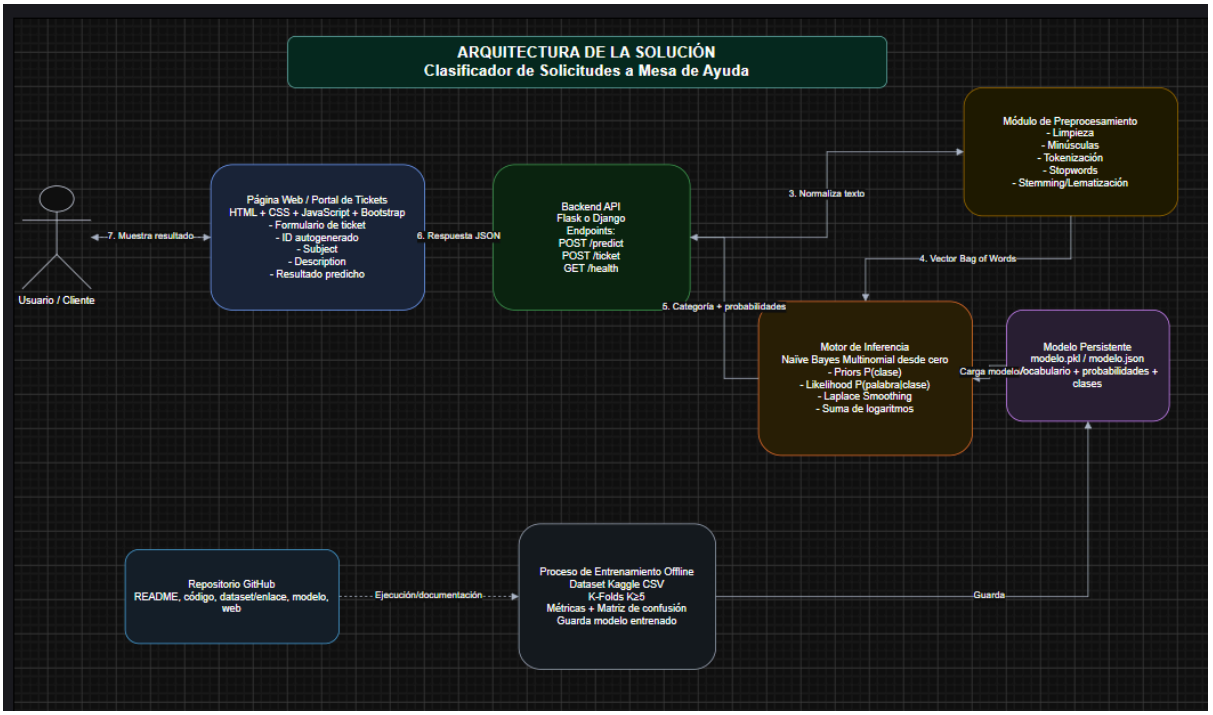
Para poder evaluar el desempeño del modelo de manera rigurosa, lo que se hizo fue implementar K-Folds Cross Validation, con un valor de K=5, esto lo que hace es que divide el dataset, de las 26,872 instancias que se tienen, se dividen en 5 particiones. En cada una de las iteración, el modelo se entrenaba con 4 particiones, y se evaluaba con la restante, rotando así el fold de prueba en cada una de las veces que se repetía el proceso, los resultados que se obtuvieron mediante este proceso fueron los siguientes:

Fold	Accuracy	F1-Score
1	0.992	0.992
2	0.991	0.991
3	0.991	0.990
4	0.991	0.991
5	0.992	0.992
Promedio	**0.991**	**0.991**

La variabilidad obtenida es mínima entre los 5 folds, teniendo una desviación estándar de $\pm\ 0.001$, tanto en Accuracy como en F1-Score.
Esto indica que el modelo es estable y consistente, es decir, no depende de una partición específica del dataset para obtener buenos resultados, lo cual es una señal positiva de que el modelo generaliza bien ante datos no vistos.

Diagramas

Arquitectura de la solución (motor de inferencia + página web).



Diagramas de Casos de Uso.

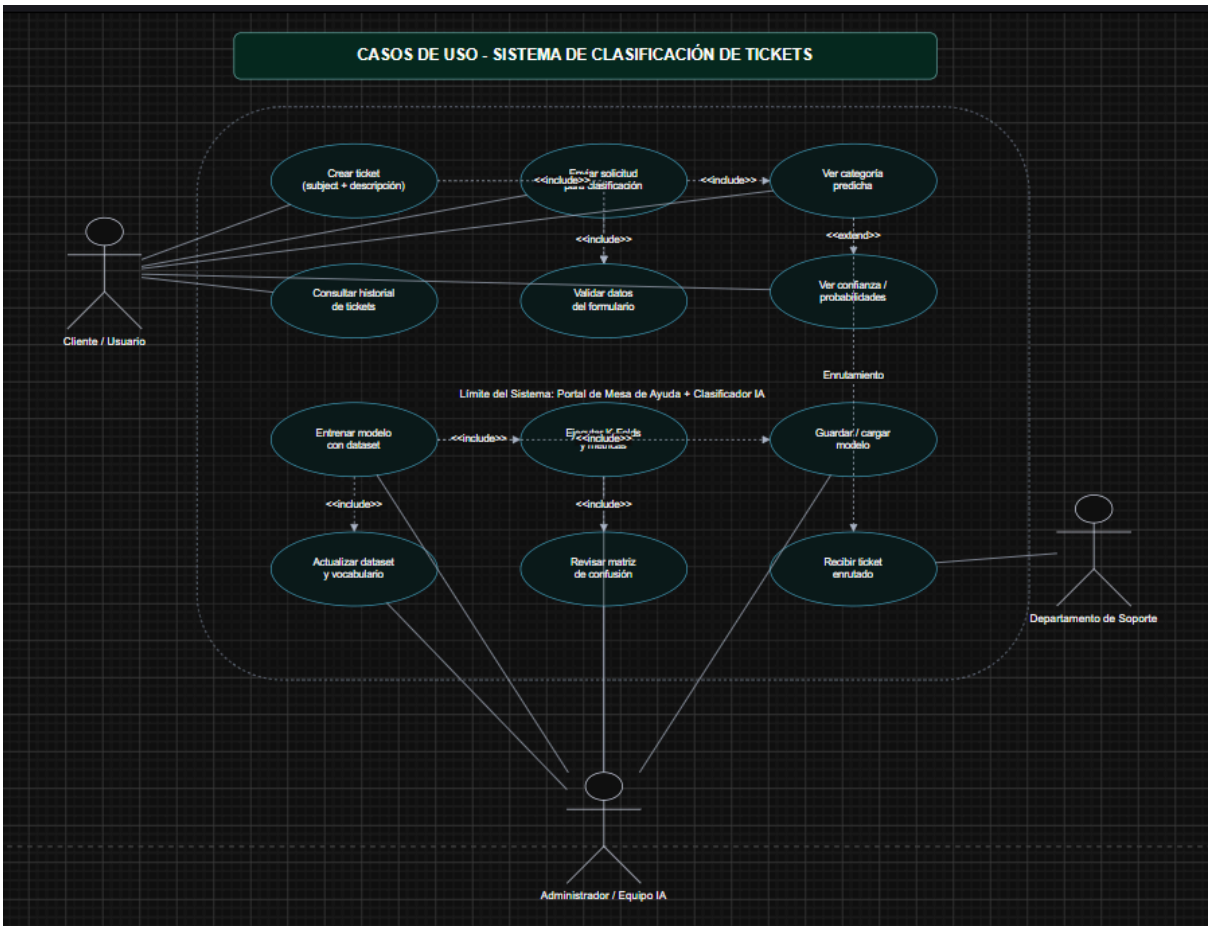


Diagrama de Flujo General.

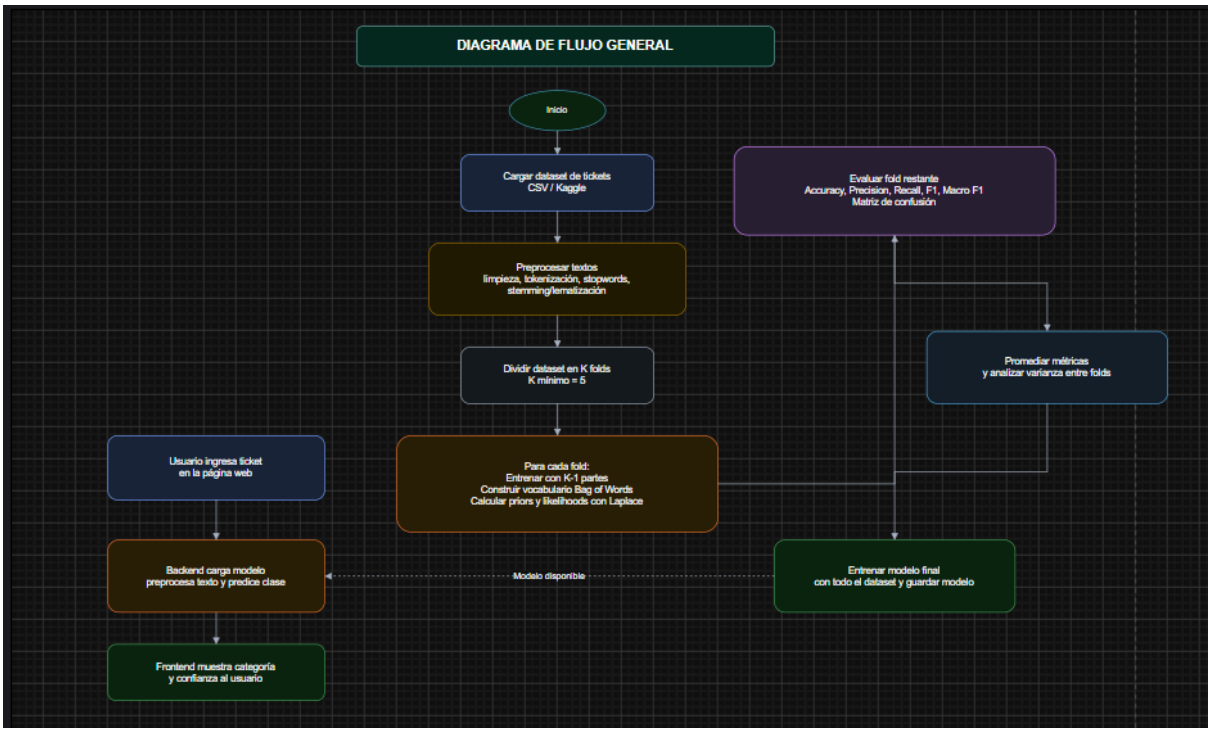


Diagrama de Componentes.

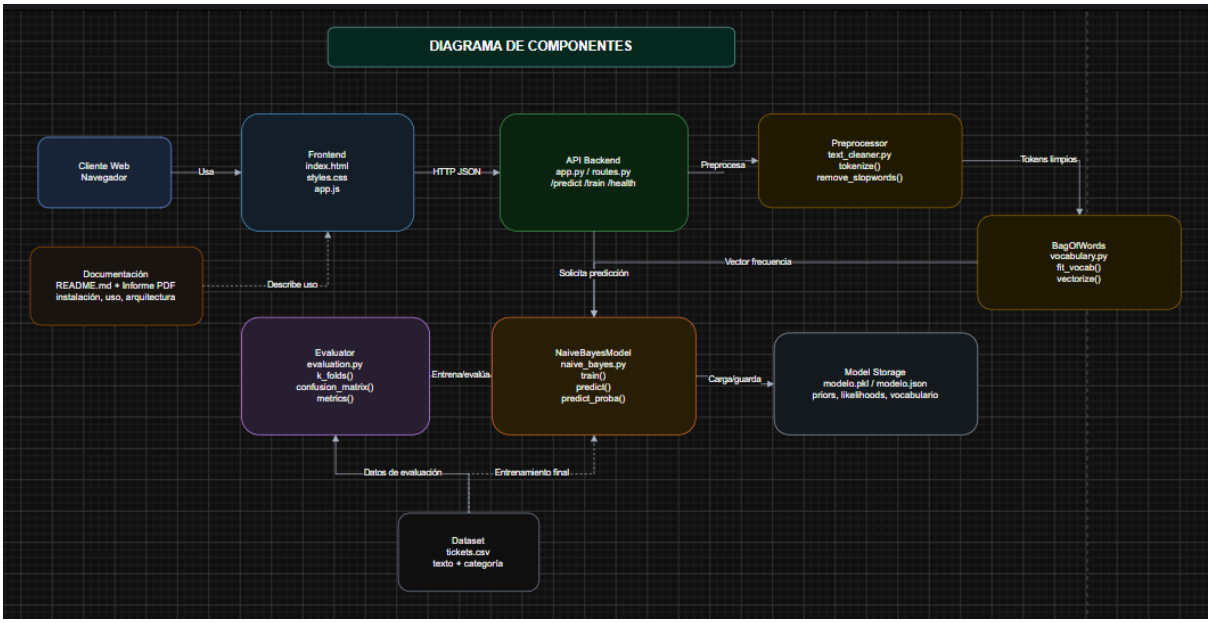
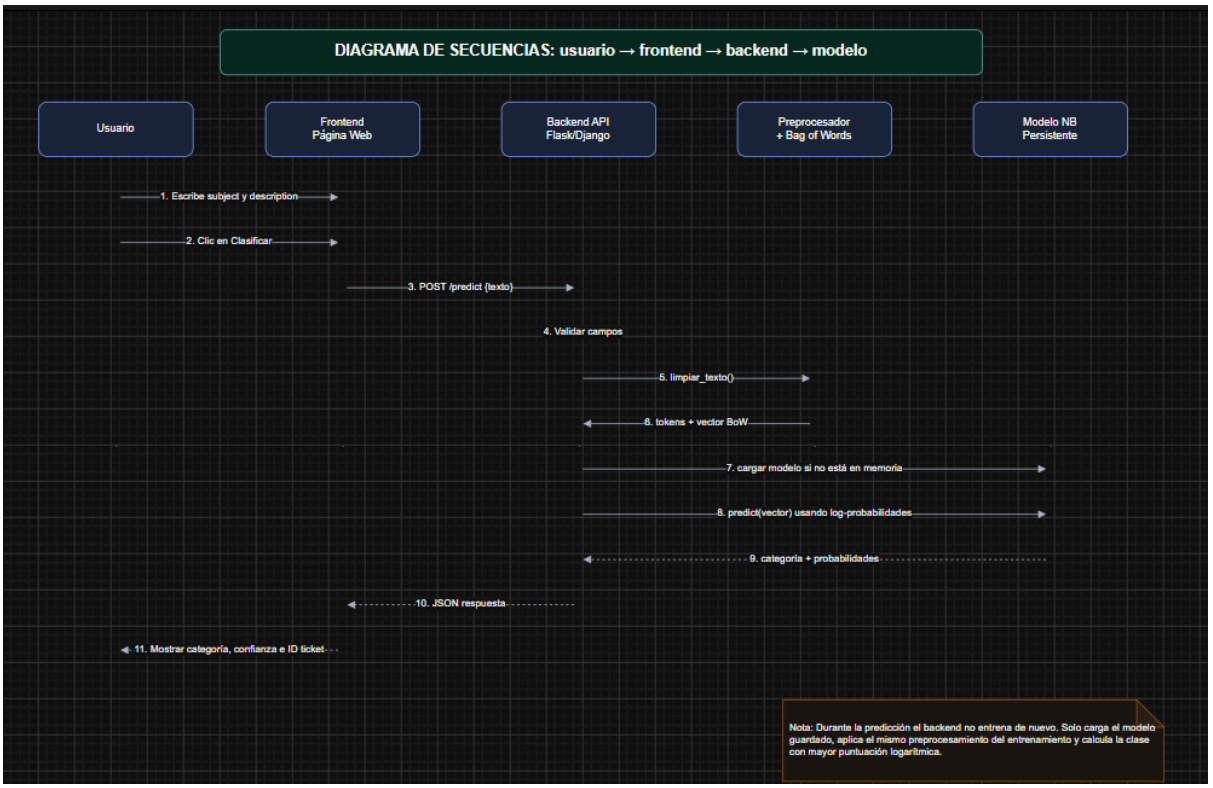
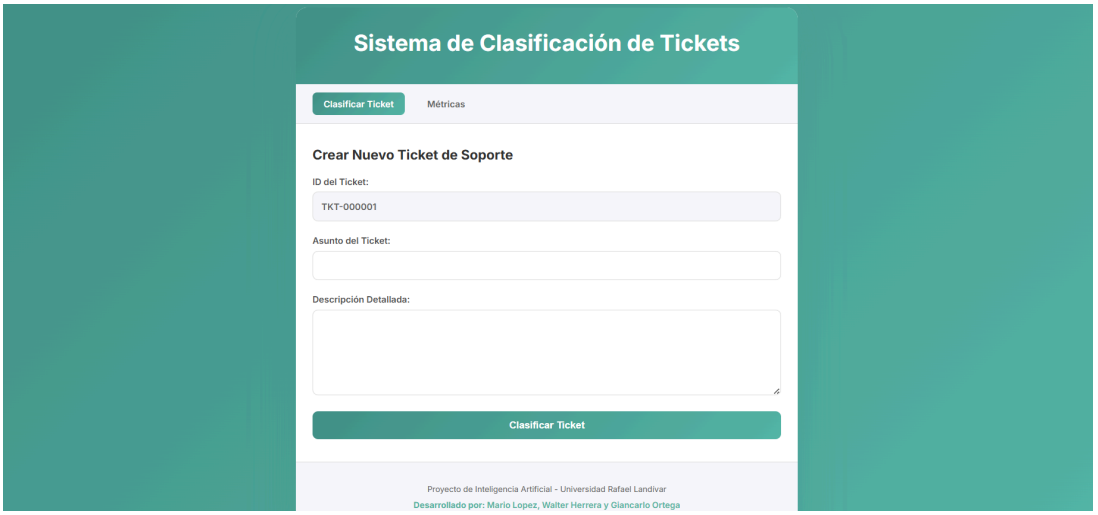


Diagrama de Secuencias (usuario → frontend → backend → modelo).



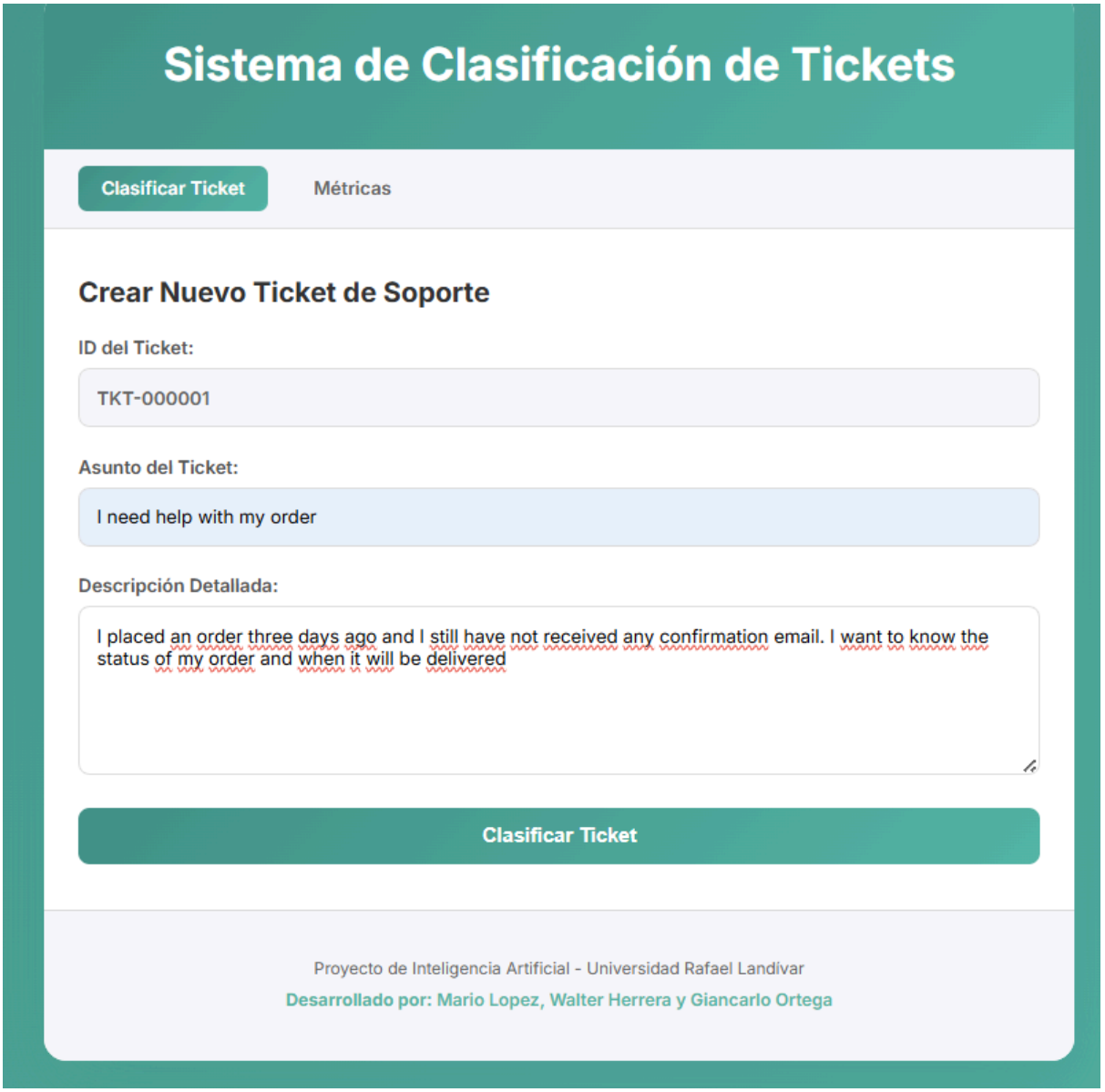
Evidencias de funcionamiento

Luego de ya tener listo el train y todo los datos del backend, ejecutamos el comando: python app.py, y automáticamente en consola se desplegará el enlace del servidor donde ya se podrá ingresar a la página web, se verá tal que así:



Ahí ya el usuario puede proceder a escribir su inconveniente o lo que sea que tenga de problema, para que así el modelo lo interprete y lo envíe al departamento correspondiente, pudiendo así ayudarle a resolverlo.

Por lo que el usuario debe de asignar el asunto del ticket, y luego ya en la descripción se puede extender y especificar qué sucede. Ya al momento de tener esas dos cosas listas, al darle click al botón de clasificar ticket, el modelo lo que hará es procesar el texto identificando la categoría correspondiente.



Como se puede observar, el sistema muestra la categoría asignada, el departamento al que fue categorizado y mandado el ticket, el nivel de confianza del modelo y las probabilidades por cada categoría disponible, mostrando gran nivel de confianza, se puede apreciar en la siguiente imagen

Clasificar Ticket Métricas

Resultado de la Clasificación

Ticket ID: TKT-000001

Categoría Asignada: **ORDER**

Confianza del Modelo: **98.5%**

Enrutado a: Departamento de Pedidos

Probabilidades por Categoría

ACCOUNT	0.0%
CANCEL	0.0%
CONTACT	0.0%
DELIVERY	1.5%
FEEDBACK	0.0%
INVOICE	0.0%
ORDER	98.5%
PAYMENT	0.0%
REFUND	0.0%
SHIPPING	0.0%
SUBSCRIPTION	0.0%

Conclusiones

- El desarrollo de este proyecto permitió comprobar que el algoritmo de Naïve Bayes Multinomial, implementado completamente desde cero, es capaz de alcanzar un rendimiento sobresaliente en tareas de clasificación de texto, obteniendo un Accuracy global de 99.15% y un Macro F1-Score de 99.11% sobre un dataset de 26,872 solicitudes distribuidas en 11 categorías.

- La estabilidad del modelo quedó evidenciada a través de la validación cruzada con K=5, donde la desviación estándar entre folds fue de apenas ±0.001, confirmando que el modelo generaliza correctamente y no depende de una partición específica para obtener buenos resultados.

- Finalmente, la integración del motor de inferencia con la interfaz web en Flask demostró que es posible llevar un modelo de clasificación a un entorno funcional y accesible para el usuario final, simulando efectivamente un sistema real de enrutamiento automático de tickets.

Aprendizajes

- El principal aprendizaje fue entender mejor cómo funciona por dentro el algoritmo de Naïve Bayes. El hecho de implementarlo desde cero obligó a comprender cada paso del proceso, incluyendo conceptos como el Laplace Smoothing y el uso de logaritmos para evitar problemas numéricos.

- También se aprendió lo importante que es el preprocesamiento en proyectos de clasificación de texto, ya que acciones como eliminar placeholders, aplicar stemming o filtrar palabras muy cortas influyen directamente en qué tan bien funciona el modelo.

- Por último, el proyecto dejó claro que la calidad del dataset es clave. No basta con tener un buen algoritmo; si los datos no están bien etiquetados o no son consistentes, el modelo difícilmente va a aprender patrones realmente útiles.